

PyTorch Notes: Improving Training Pipeline with `torch.nn` and `torch.optim`

Instructor: Nitish

Goal: Replace manual neural network code with PyTorch's built-in modules for efficiency.

Topic 1: Introduction to `torch.nn` Module

In-depth Explanation

`torch.nn` is PyTorch's core library for building neural networks. It provides:

- **Prebuilt layers** (Linear, Conv2d, LSTM, etc.)
- **Activation functions** (ReLU, Sigmoid, Tanh, etc.)
- **Loss functions** (BCELoss, CrossEntropyLoss, MSELoss, etc.)
- **Containers** (Sequential, ModuleList) to group layers
- **Utility functions** (Dropout, BatchNorm, etc.)

Without `torch.nn`, you would have to manually define weights, biases, and their interactions – tedious for large networks. With `nn`, you just specify layer sizes and activations; the module handles the rest.

Code Example: Importing and Basic Usage

```
import torch
import torch.nn as nn

# Check if module is available
print(nn.__doc__)    # prints description
```

Output (conceptual)

No specific output, but you can now use all neural network building blocks.

Topic 2: Creating a Neural Network with `nn.Module` (Single Neuron)

In-depth Explanation

To create a custom model, you **inherit from `nn.Module`**. Inside `__init__`, you define layers as class attributes. Inside `forward`, you define how data passes through layers.

Key steps:

1. `class MyModel(nn.Module):`
2. Call `super().__init__()` inside `__init__` (invokes parent constructor)
3. Define layers: `self.linear = nn.Linear(input_size, output_size)`
4. Define activation: `self.sigmoid = nn.Sigmoid()`
5. In `forward(self, x)`: pass `x` through layers sequentially and return output

Why `super().__init__()`? It registers all parameters so PyTorch can track gradients and manage them.

Code Example: Single Neuron (5 inputs → 1 output with Sigmoid)

```
import torch
import torch.nn as nn

class SingleNeuron(nn.Module):
    def __init__(self, num_features):
        super().__init__()           # Must call parent
        constructor
        self.linear = nn.Linear(num_features, 1)  # Linear layer:  $Wx$ 
        + b
        self.sigmoid = nn.Sigmoid()             # Activation

    def forward(self, x):
        out = self.linear(x)    # compute  $z = Wx + b$ 
        out = self.sigmoid(out) # apply sigmoid
        return out

# Create fake dataset: 10 samples, 5 features each
features = torch.randn(10, 5)

# Instantiate model
model = SingleNeuron(num_features=5)

# Forward pass
output = model(features)    # Note: call model directly, not
                             model.forward()
print("Output shape:", output.shape)
```

Output

```
Output shape: torch.Size([10, 1])
```

Each of the 10 samples gives a single probability value (between 0 and 1).

Inspecting Parameters

```
# View weights (shape: 1 x 5)
print("Weights:", model.linear.weight)
# View bias (shape: 1)
print("Bias:", model.linear.bias)
```

Output example:

```
Weights: Parameter containing:
tensor([[ -0.1234,  0.5678, -0.9012,  0.3456, -0.7890]],
requires_grad=True)
Bias: Parameter containing:
tensor([0.1234], requires_grad=True)
```

Total trainable parameters = 5 weights + 1 bias = **6**.

Topic 3: Adding a Hidden Layer (Multi-Layer Network)

In-depth Explanation

A hidden layer sits between input and output. For binary classification, common architecture:

- Input layer: 5 features
- Hidden layer: 3 neurons with ReLU activation
- Output layer: 1 neuron with Sigmoid activation

Parameter count:

- Hidden layer weights: $5 \times 3 = 15$
- Hidden layer biases: 3
- Output layer weights: $3 \times 1 = 3$
- Output layer bias: 1
- **Total = 22 trainable parameters**

Code Example: Network with One Hidden Layer

```
class TwoLayerNet(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.hidden = nn.Linear(num_features, 3)  # hidden layer: 5 → 3
        self.relu = nn.ReLU()  # activation for hidden
        self.output = nn.Linear(3, 1)  # output layer: 3 → 1
        self.sigmoid = nn.Sigmoid()  # activation for output

    def forward(self, x):
        x = self.hidden(x)
        x = self.relu(x)
        x = self.output(x)
        x = self.sigmoid(x)
```

```

        return x

model = TwoLayerNet(num_features=5)
output = model(features) # features shape: (10,5)
print("Output shape:", output.shape)

```

Output

```
Output shape: torch.Size([10, 1])
```

Visualizing Network Structure (optional)

```

!pip install torchinfo # if not installed
from torchinfo import summary
summary(model, input_size=(10, 5))

```

Output:

```

=====
Layer (type)          Output Shape          Param #
=====
Hidden                (10, 3)               18      (15 weights + 3
biases)
ReLU                  (10, 3)               0
Output                (10, 1)               4      (3 weights + 1 bias)
Sigmoid               (10, 1)               0
=====
Total params: 22
Trainable params: 22
Non-trainable params: 0
=====

```

Topic 4: Using `nn.Sequential` Container

In-depth Explanation

Writing `forward` manually becomes cumbersome for deep networks. `nn.Sequential` is a container that automatically chains layers. You pass layers in order, and `forward` calls them sequentially.

Benefits:

- Less boilerplate code
- Easy to read architecture
- Still inherits from `nn.Module`

Code Example: Same Two-Layer Net with Sequential

```
class TwoLayerNetSequential(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(num_features, 3),
            nn.ReLU(),
            nn.Linear(3, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.network(x)  # just one line!

model = TwoLayerNetSequential(num_features=5)
output = model(features)
print("Output shape:", output.shape)
```

Output

```
Output shape: torch.Size([10, 1])
```

Identical to previous model.

Topic 5: Using Built-in Loss Function (BCELoss)

In-depth Explanation

In manual code, you wrote loss functions yourself (e.g., binary cross-entropy). PyTorch provides **nn.BCELoss** (Binary Cross Entropy Loss) for binary classification. It expects:

- Predictions: probabilities between 0 and 1 (after sigmoid)
- Targets: 0 or 1 (same shape as predictions)

Advantage: No need to implement math, handles numerical stability automatically.

Code Example: Replacing Manual Loss with BCELoss

```
# Assume we have data
# x_train shape: (n_samples, n_features)
# y_train shape: (n_samples,) or (n_samples, 1)

# Instantiate loss function
criterion = nn.BCELoss()

# Inside training loop:
y_pred = model(x_train)  # shape: (n_samples, 1)
```

```
y_train = y_train.view(-1, 1)      # reshape to (n_samples, 1)
loss = criterion(y_pred, y_train)  # computes binary cross entropy
```

Important: BCELoss expects both inputs to have same shape. If `y_train` is 1D, use `.view(-1, 1)` to make it 2D.

Topic 6: Using Built-in Optimizer (`torch.optim`)

In-depth Explanation

Manual gradient descent required:

```
with torch.no_grad():
    model.linear.weight -= learning_rate * model.linear.weight.grad
    model.linear.bias -= learning_rate * model.linear.bias.grad
model.linear.weight.grad.zero_()
model.linear.bias.grad.zero_()
```

`torch.optim` provides optimizers (SGD, Adam, RMSprop, etc.) that handle parameter updates automatically.

Steps:

1. Create optimizer: `optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)`
2. Inside training loop, after loss computation:
 - `optimizer.zero_grad()` # clear old gradients
 - `loss.backward()` # compute new gradients
 - `optimizer.step()` # update parameters

`model.parameters()` returns all trainable parameters (weights & biases) from all layers.

Code Example: Using SGD Optimizer

```
import torch.optim as optim

# After defining model and criterion
learning_rate = 0.01
optimizer = optim.SGD(model.parameters(), lr=learning_rate)

# Training loop
for epoch in range(epochs):
    # Forward pass
    y_pred = model(x_train)
    y_train_reshaped = y_train.view(-1, 1)
    loss = criterion(y_pred, y_train_reshaped)
```

```
# Backward pass & optimization
optimizer.zero_grad()    # reset gradients
loss.backward()          # compute gradients
optimizer.step()         # update parameters
```

No manual weight updates needed!

Complete Improved Training Pipeline (All Four Improvements)

In-depth Explanation

Combining everything:

1. **Model** defined with `nn.Sequential` (or custom `nn.Module`)
2. **Activation** built into model (Sigmoid at output)
3. **Loss** = `nn.BCELoss()`
4. **Optimizer** = `optim.SGD(model.parameters(), lr)`

This is the standard PyTorch way – clean, readable, and scalable.

Code Example: Full Binary Classification Pipeline

```
import torch
import torch.nn as nn
import torch.optim as optim

# ----- 1. Load & preprocess data (example) -----
# Assume x_train, y_train are already tensors
# x_train shape: (n_samples, n_features)
# y_train shape: (n_samples,) with values 0/1

n_features = x_train.shape[1]

# ----- 2. Define Model -----
model = nn.Sequential(
    nn.Linear(n_features, 3),
    nn.ReLU(),
    nn.Linear(3, 1),
    nn.Sigmoid()
)

# ----- 3. Define Loss -----
criterion = nn.BCELoss()

# ----- 4. Define Optimizer -----
learning_rate = 0.01
```

```

optimizer = optim.SGD(model.parameters(), lr=learning_rate)

# ----- 5. Training Loop -----
epochs = 100
for epoch in range(epochs):
    # Forward pass
    y_pred = model(x_train)
    y_train_reshaped = y_train.view(-1, 1) # ensure same shape
    loss = criterion(y_pred, y_train_reshaped)

    # Backward pass & optimization
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # Optional: print loss every 10 epochs
    if (epoch+1) % 10 == 0:
        print(f"Epoch {epoch+1}/{epochs}, Loss: {loss.item():.4f}")

# ----- 6. Evaluation -----
with torch.no_grad():
    y_pred_final = model(x_train)
    predicted = (y_pred_final > 0.5).float()
    accuracy = (predicted.view(-1) == y_train).float().mean()
    print(f"Accuracy: {accuracy.item():.4f}")

```

Sample Output

```

Epoch 10/100, Loss: 0.6932
Epoch 20/100, Loss: 0.6541
Epoch 30/100, Loss: 0.6123
...
Epoch 100/100, Loss: 0.3215
Accuracy: 0.8675

```

Summary of Four Improvements

Manual Approach	PyTorch Built-in
Define weights & biases manually	<code>nn.Linear(in, out)</code>
Write activation function code	<code>nn.Sigmoid()</code> , <code>nn.ReLU()</code>
Implement loss formula	<code>nn.BCELoss()</code> , <code>nn.CrossEntropyLoss()</code>
Manual gradient descent updates	<code>optim.SGD() + .step()</code>

Key Takeaways

1. **Always inherit from `nn.Module`** for custom models.
2. **Call `super().__init__()`** in your model's constructor.
3. **Use `nn.Sequential`** for simple feed-forward networks.
4. **Never call `.forward()` directly** – call the model instance.
5. **Loss functions** are callable objects; ensure input shapes match.
6. **Optimizer** requires `model.parameters()` and learning rate.
7. **Training loop pattern:** `zero_grad` → `backward` → `step`.

These notes cover everything Nitish explained. Write them in your copy with headings and code blocks. Happy learning!

Notes on PyTorch Dataset & DataLoader Classes

1. Problem with Batch Gradient Descent

In batch gradient descent, the entire dataset is used at once to compute gradients and update parameters.

Problems:

- **Memory inefficient:** Entire dataset must fit in RAM (e.g., 10GB of images cannot be loaded at once).
- **Slow convergence:** Parameters update only once per epoch; updates are infrequent and slow.

Solution: Mini-batch gradient descent – split data into smaller batches, update parameters after each batch.

2. Mini-Batch Gradient Descent

- Divide dataset into batches (e.g., 1000 rows → 10 batches of 100 rows each).
- For each batch: forward pass → loss → backward pass → update parameters.
- Repeat for all batches → one epoch completed.
- Faster convergence and memory efficient.

3. Problems with Manual Mini-Batch Implementation

Manually creating batches using loops has several issues:

Problem	Description
No standard interface	Data loading logic (e.g., reading images from folders) has no dedicated place.
No easy transformations	Cannot apply resizing, normalization, etc., before